

MICRO-SEGMENTED CLOUD NETWORK ARCHITECTURE USING OPEN-SOURCE TOOLS

AWS + Terraform + Suricata

Project Report & Interview Explanation Guide

Mana language: Telugu + English mix

Objective: Build a secure three-tier AWS network where Web, Application, and Database layers are separated using private/public subnets, routing, and least-privilege Security Groups. Add Suricata as an open-source IDS and use network traffic monitoring to detect suspicious activity.

Core architecture: Internet Public Web Private App Isolated RDS Database

1. Project Overview

Ee project main aim enti ante AWS lo secure ga oka three-tier application network build cheyyadam. Normal ga anni resources okate network lo unte unnecessary access perigipothundi. Manam Web, App, DB layers ni separate chesi, e layer ki e layer access undalo strict ga define chesam.

Simple ga: Web server public side lo untundi. Application servers private subnet lo untayi. Database completely isolated private DB subnet lo untundi. Web direct ga DB ni access cheyyakudadhu; App matrame DB ki PostgreSQL 5432 port lo connect avvali.

2. Technologies Used

- AWS VPC – complete isolated cloud network create cheyyadaniki.
- Terraform – infrastructure ni code dwara repeatable ga provision cheyyadaniki.
- EC2 – Web, App, and IDS workloads run cheyyadaniki.
- RDS PostgreSQL – managed relational database.
- Security Groups – layer-to-layer access control.
- Internet Gateway – public subnet internet connectivity.
- NAT Gateway – private App servers outbound internet access kosam.
- IAM + Systems Manager – secure EC2 administration without depending on public SSH.
- Suricata – open-source network intrusion detection.
- VPC Traffic Mirroring – selected ENI traffic ni IDS side ki mirror cheyyadaniki.

3. High-Level Architecture

```
INTERNET
|
Internet Gateway
|
+-----+-----+
| |
Public Web IDS / Security
Subnets Monitoring
| |
Web EC2 Suricata IDS
|
TCP 8080
|
+-----+-----+
| |
Private App 1 Private App 2
| |
+-----+-----+
|
TCP 5432
|
Private DB Subnets
|
RDS PostgreSQL
```

Architecture meaning: Public side lo user request Web server daggariki vastundi. Web only Application layer ni 8080 lo access chestundi. Application layer only Database ni 5432 lo access chestundi. Database ki direct internet route ledu.

4. VPC & Subnet Design

VPC CIDR: 10.0.0.0/16. Two Availability Zones use chesam for better availability.

Tier	Subnet 1	Subnet 2	Purpose
Public Web	10.0.1.0/24	10.0.2.0/24	Internet-facing resources
Private App	10.0.11.0/24	10.0.12.0/24	Application servers
Private DB	10.0.21.0/24	10.0.22.0/24	RDS/database layer

Mana explanation: Public subnet lo public IP unna Web EC2 undi. App subnet private kabatti direct internet nunchi access cheyyalem. DB subnet inka strict: default internet route kuda ledu.

5. Routing Architecture

- Public Route Table: 0.0.0.0/0 Internet Gateway.
- Private App Route Table: 0.0.0.0/0 NAT Gateway.
- Private DB Route Table: no default internet route.

NAT Gateway enduku? App servers ki packages updates/install cheyyali ante outbound internet kavachu. Kani internet nunchi App server ki new inbound connection allow cheyyakunda NAT one-way outbound connectivity provide chestundi.

6. Security Group / Micro-Segmentation

Security Group	Inbound	Source
Web SG	TCP 80, 443	Internet
App SG	TCP 8080	Web SG only
DB SG	TCP 5432	App SG only

Core concept: App SG source ga Web SG ni use chestundi. DB SG source ga App SG ni use chestundi. IP addresses ni broad ga allow cheyyakunda security-group-to-security-group rules use cheyyadam least-privilege approach.

```
Internet -> Web :80/443 ALLOWED
Web -> App :8080 ALLOWED
App -> RDS :5432 ALLOWED
Web -> RDS :5432 BLOCKED
Internet -> App BLOCKED
Internet -> RDS BLOCKED
```

7. Terraform Modular Design

```
micro-project/
  ├── main.tf
  ├── providers.tf
  ├── versions.tf
  ├── variables.tf
  ├── terraform.tfvars
  ├── outputs.tf
  ├── .gitignore
  ├── modules/
  │   ├── vpc/
  │   ├── security-groups/
  │   ├── ec2/
  │   └── rds/
```

Terraform root module orchestration chestundi. VPC module networking create chestundi; Security Groups module access rules create chestundi; EC2 module Web/App/IDS instances create chestundi; RDS module PostgreSQL create chestundi.

Mana language lo: Oka huge main.tf lo anni rayakunda modules ga divide chesam. Reuse cheyyadam, maintain cheyyadam, troubleshooting easy avvadam kosam idi useful.

8. Web Tier

Web EC2 public subnet lo deploy ayyi Apache/httpd run chestundi. User request first Web tier ki vastundi.

```
dnf install -y httpd
systemctl enable httpd
systemctl start httpd
```

Browser nunchi Web public IP ki request chesthe Web page response istundi.

9. Application Tier

Two private App EC2 instances deploy chesam: one in each AZ. Simple Python HTTP application TCP 8080 meeda run chestundi.

```
curl http://10.0.11.198:8080
curl http://10.0.12.206:8080
```

Both App servers response ivvadam valla Web/App network path working ani verify chesam.

10. Database Tier – RDS PostgreSQL

RDS PostgreSQL private DB subnets lo deploy chesam. Database name microdb, port 5432. Master username is stored securely and should never be committed to Git.

App server nunchi PostgreSQL connection successful ga establish chesam. DNS endpoint private IP ki resolve ayyindi, and TCP 5432 connectivity successful.

```
psql -h <RDS-ENDPOINT> -U <DB-USER> -d microdb
```

Meaning: App DB allowed. But Web DB direct access blocked. Idi mana micro-segmentation ki strongest proof.

11. Connectivity Validation

Test	Expected	Observed
Web App TCP 8080	Allow	PASS
App RDS TCP 5432	Allow	PASS
Web RDS TCP 5432	Block	PASS
RDS public access	Block	PASS

Web DB blocked test lo TCP 5432 connection fail ayyindi. App DB test lo TCP 5432 SUCCESS vachindi. Therefore access path intentionally restricted.

12. IAM & Systems Manager

EC2 instances ki AmazonSSMManagedInstanceCore policy attach chesi Systems Manager use chesam. Dini valla instance management kosam public SSH access meeda complete dependency undadu.

Simple ga: Server ni manage cheyyadaniki direct internet-facing SSH open cheyyakunda SSM Session Manager use cheyyachu.

13. Suricata IDS

Suricata 8.0.1 source nunchi build chesi install chesam because Amazon Linux 2023 default repositories lo required package direct ga available kaaledu. Dependencies install chesi source compile chesam.

Suricata rules kosam Emerging Threats Open rules update chesam. Configuration test successful ga complete ayyindi.

IDS instance: t3.small. Interface: ens5. Private IP: 10.0.1.138.

Important technical point: Suricata EC2 ni create cheyyadam alone entire VPC traffic ni automatically inspect cheyyadu. Actual network IDS visibility kosam VPC Traffic Mirroring or another traffic-tap architecture required.

14. IDS Architecture with Traffic Mirroring

```
Traffic Source ENI
(Web / selected workload)
|
| mirrored packets
v
VPC Traffic Mirroring
|
v
IDS / Target ENI
|
v
Suricata
|
v
eve.json / alerts
```

Mana project lo Suricata is the detection engine. Traffic Mirroring selected network traffic ni IDS side ki send chestundi. Suricata packets inspect chesi matching rules unte events generate chestundi.

15. Suricata Detection Flow

```
Network traffic
↓
Traffic Mirror
↓
Suricata packet capture
↓
Rules evaluation
↓
Alert / event
↓
eve.json / logs
```

eve.json structured JSON format lo security events store cheyyadaniki useful. Interview lo 'Suricata detects suspicious patterns based on rules and writes structured events to logs' ani cheppachu.

16. Controlled Security Testing

Own infrastructure meeda safe, controlled tests perform cheyyali. Examples: connectivity tests, controlled port scanning, suspicious HTTP patterns, and log verification. Destructive attacks or third-party targets test cheyyakudadhu.

Expected result: test traffic mirror Suricata alert/event log.

17. Final Terraform Validation

```
terraform fmt -recursive
terraform validate
terraform plan
```

Final plan ideally 'No changes' chupiste Terraform state and configuration aligned unnayi ani meaning.

18. GitHub Security Cleanup

Credentials accidental ga repository lo commit avvakudadhu.

```
.terraform/
terraform.tfstate
terraform.tfstate.*
*.tfvars
*.tfvars.json
```

```
*.pem
*.key
crash.log
crash.*.log
```

Especially terraform.tfvars to database password unte GitHub ki push cheyyakudadhu. Previously exposed credential unte final project publish cheyyadaniki mundu rotate/change cheyyali.

19. Project Outcome

- Three-tier AWS architecture successfully implemented.
- Public Web, private App, and isolated DB tiers separated.
- Least-privilege Security Groups implemented.
- Web App allowed; App RDS allowed; Web RDS blocked.
- RDS PostgreSQL deployed privately.
- EC2 management supported through SSM.
- Suricata IDS installed, configured, and rules loaded.
- Traffic Mirroring provides the network visibility required for proper IDS inspection.
- Terraform provides repeatable Infrastructure as Code.

20. Interview Explanation – 60 Seconds

“My project is a Micro-Segmented Cloud Network Architecture using AWS, Terraform and Suricata. I designed a three-tier VPC with public Web, private Application and isolated Database subnets across two Availability Zones. I used Security Groups with least-privilege rules so that Web can communicate with App only on TCP 8080, and App can communicate with PostgreSQL RDS only on TCP 5432. Direct Web-to-Database access is blocked. I automated the infrastructure using reusable Terraform modules and used IAM with Systems Manager for secure EC2 administration. For network security monitoring, I deployed Suricata as an open-source IDS and used traffic mirroring to provide selected network traffic for inspection. I validated the architecture with positive and negative connectivity tests.”

21. Interview Questions & Simple Answers

Q: Why micro-segmentation?

A: One compromised layer should not automatically get access to every other layer. Each tier gets only the communication it needs.

Q: Why is DB private?

A: Database should not be directly reachable from the internet. It is accessed only by the application tier.

Q: Why use NAT Gateway?

A: Private App servers may need outbound internet for updates/packages without receiving unsolicited inbound internet traffic.

Q: Why use Security Group references?

A: It is cleaner and least-privilege oriented: App accepts traffic from the Web SG, and DB accepts traffic from the App SG.

Q: Why Terraform?

A: Infrastructure becomes version-controlled, repeatable, auditable, and easier to recreate.

Q: Why Suricata?

A: It is an open-source IDS capable of inspecting traffic and generating security alerts/events based on rules.

Q: Does Suricata EC2 automatically see all VPC traffic?

A: No. Network visibility must be provided using Traffic Mirroring or another suitable capture architecture.

Q: What proves micro-segmentation?

A: Web-to-App succeeds, App-to-DB succeeds, while Web-to-DB on 5432 is blocked.

22. Final Checklist

- Terraform code formatted and validated
- VPC/subnets/routes verified
- Security Group rules verified
- Web/App/RDS connectivity tested
- Web DB direct access blocked
- SSM access verified
- Suricata installed and configuration tested
- Traffic Mirroring configured and validated
- Suricata events/logs verified
- Secrets excluded from Git
- README and architecture diagram added
- AWS resources reviewed and unused resources removed to control cost

23. One-Line Project Summary

"A Terraform-driven AWS three-tier micro-segmented network with least-privilege access controls, private RDS, secure EC2 management, and Suricata-based network intrusion detection."